



HSE INCIDENT & CAPA COMPLIANCE PORTAL

Business Requirements Document (BRD) - Platform Specification

DKSH HSE Compliance Division

Author: Regional HSE Compliance Team & Business Analysts

Date: July 2026 | Version: 2.1 | Status: APPROVED

Confidentiality Notice: This document contains proprietary operational specs for internal use inside DKSH.

BUSINESS REQUIREMENTS DOCUMENT (BRD)

HSE Incident & CAPA Compliance Portal

Document Control

Item	Specification Details
Document Title	Business Requirements Document (BRD) ½ Complete Site-Wide Functional & Technical Specifications
Project Name	DKSH HSE CAPA Production System
System Version	1.1.0 (Production-Ready)
Release Date	July 5, 2026
Target Audience	Executive Steering Committee, Regional HSE Directors, Software Engineering Teams, QA Teams
Document Status	**Approved for Implementation & Deployment**

1. Executive Summary & Vision

1.1 Vision Statement

The DKSH HSE Incident & CAPA Compliance Portal is designed to digitize, standardize, and streamline the tracking of Health, Safety, and Environment (HSE) occurrences and the recording of Corrective and Preventive Actions (CAPA) across the entire Asia-Pacific region. CAPA tasks and their execution are conducted and tracked on an external system, meaning this portal is utilized purely as a central register for HSE incident and CAPA logging. By replacing fragmented manual spreadsheets, the portal serves as a single, unified source of truth for safety reporting, performance analytics, and regulatory compliance.

1.2 Strategic Goals

- ½ Empower Local Teams: Enable site-level coordinators to easily report hazards, near-misses, and injuries through an interactive, intuitive multi-step wizard.
- ½ Drive Operational Accountability: Structure the investigation phase using the industry-standard 5-Whys Root Cause Analysis and track CAPA tickets from creation to verification.
- ½ Deliver Executive Transparency: Render high-level regional indicators (LTIFR, TRIR, Severity Rate) on a visual dashboard to support data-driven safety decisions.

🔒 Enforce Corporate Governance: Strictly apply Role-Based Access Control (RBAC) to segment market-specific records and prevent data tampering or leakage.

2. Role-Based Access Control (RBAC) Matrix

To secure corporate data, four distinct system roles are supported. Each role dictates read, write, edit, and configuration capabilities across modules.

Permission / Action	Reporter (Level 1)	Country HSE Manager	Regional HSE	Superuser (Global)
View Regional Dashboard	No (auto-redirected to Country view)	No (restricted to country context)	**Yes** (full regional consolidation)	**Yes** (full global consolidation)
View Country Dashboard	Yes (restricted to their default country)	Yes (restricted to their default country)	**Yes** (can toggle any regional country)	**Yes** (can toggle any country globally)
Log Incident Reports (Tickets)	**Yes** (creates and saves drafts)	**Yes** (full write permissions)	**Yes** (full write permissions)	**Yes** (full write permissions)
Conduct Investigation (Step 4)	**No** (disabled in wizard)	**Yes** (full write & verification)	**Yes** (full write & verification)	**Yes** (full write & verification)
Edit Incident Drafts	Yes (only their own drafts)	Yes (all drafts within their market)	**Yes** (all drafts in their region)	**Yes** (unrestricted editing)
Modify Labor Hours Grid	**No** (view-only)	**Yes** (authorized market only)	**Yes** (authorized region markets)	**Yes** (unrestricted saving)
Configure Masters (DCs)	**No** (view-only UI)	**No** (view-only configuration lists / restricted)	**Yes** (full regional master DC configuration access)	**Yes** (full administrative master access)
Extract Safety Data	Redacted CSV format (no PII)	Full CSV/XLSX for local market	**Yes** (Full Regional CSV & PDF Summaries)	**Yes** (Full Regional CSV/XLSX & executive PDF)

3. Module Specifications & Functional Workflows

3.1 Module 1: The HSE Performance Dashboard

The dashboard supports two display modes based on the active state:

1. Regional Dashboard (Consolidated view across all markets for Superusers)
2. Country Dashboard (Localized operational statistics with input forms)

3.1.1 Advanced Date Range Picker

- 🔗½ Highly interactive calendar popover featuring a two-panel structure:
- Left Panel (Presets): Includes `Last 30 Days`, `Current Month`, `YTD`, `Last Year`, etc.
 - Right Panel: Responsive interactive grid to manually pick a continuous range of start and end dates. Handles responsive reflowing on mobile devices (`w-[92vw]`).

3.1.2 Key Mathematical Performance Indicators (KPIs)

🔗½ Safe Work Days: Represented as a dynamic indicator displaying consecutive days without any Lost Time Injury (LTI) occurrences.

🔗½ Corporate Standard Frequency Rate: The portal normalizes incident frequency rates per 1,000,000 hours worked:

$$\text{Standard Rate} = \frac{\text{Total Incident Cases of a Specific Type} \times 1,000,000}{\text{Total Labor Hours Worked}}$$

🔗½ Standardized Calculations List:

1. Fatalities Rate (DEATH):
$$\text{DEATH Rate} = \frac{\text{Total Fatalities (No.7)} \times 1,000,000}{\text{Total Labor Hours Worked}}$$
2. High Consequence Work-Related Injury Rate (>6M):
$$\text{>6M Rate} = \frac{\text{Total High-Consequence Work-Related Injuries (No.8)} \times 1,000,000}{\text{Total Labor Hours Worked}}$$
3. Lost Time Injury Rate (LTI):
$$\text{LTI Rate} = \frac{\text{Total Lost Time Injuries (No.9)} \times 1,000,000}{\text{Total Labor Hours Worked}}$$
4. Recordable Injury Rate (REC):
$$\text{REC Rate} = \frac{\text{Total Recordable Work-Related Injuries (No.8-13)} \times 1,000,000}{\text{Total Labor Hours Worked}}$$
5. Work-Related Injury Rate (WRI):
$$\text{WRI Rate} = \frac{\text{Total Work-Related Injuries (No.8-14)} \times 1,000,000}{\text{Total Labor Hours Worked}}$$
6. Property Damage Case Count (PD): Absolute count of property damage incidents (No.15).
7. Near Misses Case Count (NM): Absolute count of near-miss occurrences (No.16).
8. Hazard Observation Case Count (HZ): Absolute count of hazard observations (No.17).

3.1.3 Interactive Labor Hours Configurator (Excel-Style Grid)

🔗½ Grid Layout: Monthly matrix dividing labor categories into Employee Hours and Other/Contractor Hours.

🔗½ Calculations: Sum totals are generated in real-time.

🔗½ Lock & Save Security: Input fields are strictly disabled for the `Reporter` role. `Level2` and `Superuser` roles can input numbers and click "Save & Unlock" to write directly to database state (simulated via LocalStorage).

Fields involved in Module 1:

No.	Field Name	Field Type	Mandatory	Description /
1	**Date Range Picker Presets**	Button Selection Group	No	Quick selector options (e.g., `Last 30 Days`, `Current Month`, `YTD`, `Last Year`).

2	**Custom Start Date**	Calendar Date Picker	Yes (if Custom selected)	Beginning boundary date for safety KPI analytics.
3	**Custom End Date**	Calendar Date Picker	Yes (if Custom selected)	Ending boundary date for safety KPI analytics.
4	**Dashboard Country Filter**	Dropdown Selector	Yes (for admin/regional)	Filters safety visualizers/metrics to the selected market. Locked to local context for standard users.
5	**Labor Year Selector**	Dropdown Selector	Yes	Selects target calendar year for hours entry (e.g. 2026).
6	**Labor Month Selector**	Dropdown Selector	Yes	Selects target calendar month for hours entry.
7	**Employee Hours**	Numeric Input	Yes	Sum of direct employee exposure hours worked. Supports decimals.
8	**Contractor / Other Hours**	Numeric Input	Yes	Sum of external contractor/subcontractor exposure hours. Supports decimals.

3.2 Module 2: Step-by-Step Reporting Wizard (InspectionForm)

Designed as a structured multi-step wizard to guide users from initial notification to official closure.

INSPECTION FORM WIZARD			
Step 1: Info	Step 2: Rosters	Step 3: Body / Class	Step 4: Root Cause (RW)
- Date, Time, site	- Add Personnel	- Click body part	- 5-Whys systematic checklist
- Choose category	- Flag employee/sub	- Check 17-points	- Action items & sign-off
- CC notification	- Injured / witness	- Select injuries	- Verification & closure

Step 1: Basic Operational Data

🔗Fields: Date, Time, Category Select (`Injury`, `Ill-health`, `Property damaged`, `Near miss`, `Hazard Observation`), Location/Distribution Center, and CC Emails.

🔗Behavior: The location field displays a filtered list of only active Distribution Centers registered for the selected country.

Fields involved in Step 1 (Basic Info):

No.	Field Name	Field Type	Mandatory	Description /
-----	------------	------------	-----------	---------------

1	**Reporting Date**	Calendar Date Picker	Yes	The date when the safety occurrence took place (past or current date).
2	**Reporting Time**	Time Input (HH:MM)	Yes	The clock time of the safety occurrence.
3	**Incident Category**	Dropdown Selector	Yes	Choice of: `Injury`, `Ill-health`, `Property damaged`, `Near miss`, `Hazard Observation`.
4	**Distribution Center / Site**	Dropdown Selector	Yes	Active registered DC/facility. Filtered dynamically based on active country context.
5	**CC Notification Emails**	Text Area / Email list	No	Semi-colon separated list of stakeholder email addresses to auto-notify.

Step 2: Involved Personnel & Witness Rosters

Dynamic entry grid to add stakeholders into two tables:

1. Involved/Injured Personnel Table: Captures Name, Staff ID, Department, Business Unit, Employee Status (Direct vs Contractor), Injury Flag (Injured vs Involved), and Location within the plant.
2. Witness Roster Records: Captures Witness Name, Staff ID, Department, Business Unit, and Employment type.

Fields involved in Step 2 (Rosters):

No.	Field Name	Field Type	Mandatory	Description /
1	**Involved Person Name**	Text Input	Yes (if row added)	Full name of the involved, witness, or injured worker.
2	**Involved Staff ID**	Alphanumeric Text	Yes (if row added)	Employee system identifier or contractor badge number.
3	**Involved Department**	Text Input	Yes (if row added)	Standard operations department of the worker.
4	**Involved Business Unit**	Dropdown Selector	Yes (if row added)	Operating commercial BU of the worker.
5	**Involved Employee Status**	Dropdown Selector	Yes (if row added)	Choices: `Direct Employee` or `Contractor`.
6	**Involved Injury Designation**	Dropdown Selector	Yes (if row added)	Choices: `Injured` (undergoes body mapping) or `Involved`.

7	**Specific Location**	Text Input	No	Precise location of the worker at the time of incident.
8	**Witness Name**	Text Input	Yes (if witness added)	Full name of the eye witness.
9	**Witness Staff ID**	Alphanumeric Text	No	System ID of the witness.
10	**Witness Department**	Text Input	No	Operations department of the witness.
11	**Witness Business Unit**	Dropdown Selector	No	Business unit of the witness.
12	**Witness Employee Status**	Dropdown Selector	No	Choices: `Direct Employee` or `Contractor`.

Step 3: Injury Mapping & Classification Criteria

🔗 ½ Interactive Body Map (HumanFigure):

- Custom vector graphic representing human anatomy from front and back profiles.
- Users can click on anatomical nodes (e.g., Head, Face, Eye, Shoulder, Hands, Finger, Knee, Feet) to toggle injury status.
- Highlighted parts turn red with a pulsing radar ring to indicate an injury.
- Mobile-Specific Touch-First UX Refactor (max-width: 767px):
 - Decluttered UI: Left/right side textual indicators and the standard checkbox lists are completely hidden (`display: none;`) to maximize vertical breathing space and isolate the interactive graphic.
 - Centered Diagram: The central anatomical illustration is isolated, centered (`margin: 0 auto;`), and scaled cleanly to standard mobile viewports (`width: 100%; max-width: 350px;`) without horizontal scrolling.
 - Expanded Touch Targets: Invisible overlay hit circles of at least `44px x 44px` are configured for all hotspots to eliminate frustrating mis-taps on small touchscreens.
 - Boundary-Aware Tooltips: Tooltips adapt dynamically to coordinate locations. High hotspots (like the Head) render tooltips below the node rather than above, and extreme edge hotspots use horizontal offset shifts to completely eliminate text clipping or overflow against container boundaries.
 - Persistent Selection Summary (Chip List): A wrap-aligned flexbox chip list (`display: flex; flex-wrap: wrap; gap: 8px; align-items: center; min-height: 48px;`) displays active selections. Tapping the " " on a chip immediately removes it from the selection list and the radar dot on the body graphic.

🔗 ½ 17-Point Classification Criteria Checklist:

- Complete checklist determining the severity of the event (e.g., results in death, recovery more than 6 months, absence more than 1 day with Lost Time Days input, medical treatment beyond first aid, significant injury, or simple first aid).
- This systematic questionnaire automatically classifies the event severity (e.g., LTI, MTC, FAC, Near Miss) in the backend.

Fields Involved in Step 3 (Mapping & Classification):

No.	Field Name	Field Type	Mandatory	Description /
-----	------------	------------	-----------	---------------

1	**Occurrence Title**	Text Input	Yes	A short descriptive title summarizing the incident (e.g., "Maintenance Worker Sustained Fractured Leg").
2	**Event Description**	Text Area	No	Detailed narrative describing safety gaps, machine speed settings, cargo conditions, etc.
3	**Anatomical Body Parts**	Interactive Vector Area	No	Click hotspots (e.g. Head, Face, Eyes, Arm, Hand, Leg, Feet) to log injured physical locations.
4	**17-Point Criteria Checklist**	Interactive Checkboxes	Yes	Multiple-choice checkboxes mapping to severity classification standards (Items 1-17).
5	**Fatalities Count (No. 7)**	Numeric Input	Yes (if checked)	Mandatory when "Results in death" is flagged. Tracks the number of deceased persons.
6	**High-Consequence Recovery Count (No. 8)**	Numeric Input	Yes (if checked)	Mandatory when "More than 6 months till recovery" is flagged. Tracks number of persons.
7	**Absence / LTI Count (No. 9)**	Numeric Input	Yes (if checked)	Mandatory when LTI absence of more than one day is flagged. Tracks number of persons.
8	**Lost Time Days (No. 9.1)**	Numeric Input	Yes (if checked)	Mandatory when LTI absence is flagged. Calculated total work shifts lost due to injury.
9	**Restricted Work Count (No. 10)**	Numeric Input	Yes (if checked)	Tracks number of persons assigned to restricted work or change of role.
10	**Loss of Consciousness Count (No. 11)**	Numeric Input	Yes (if checked)	Tracks number of persons suffering loss of consciousness.
11	**Medical Treatment Count (No. 12)**	Numeric Input	Yes (if checked)	Tracks number of persons requiring medical treatment beyond first aid.

12	**Significant Injury Count (No. 13)**	Numeric Input	Yes (if checked)	Tracks number of persons with significant injury diagnosed by a physician.
13	**First Aid Count (No. 14)**	Numeric Input	Yes (if checked)	Tracks number of persons requiring first aid treatment only.
14	**PDPA Consent Checkbox (Reporter)**	Checkbox	Yes (for Reporter role)	Mandatory agreement to DKSH data privacy policy prior to final submission.

Step 4: Root Cause Analysis, Actions & Sign-off (HSE Managers & Superusers Only)

🔗½Role Restrictions: Level 1 Reporters are strictly restricted from visiting or submitting Step 4. Instead, their wizard consists of 3 steps, and they sign their Personal Data Protection Act (PDPA) Consent and submit their Incident Report directly at the bottom of Step 3.

🔗½5-Whys Methodology Section: Sequential input blocks to trace failure chains (Why did it happen? -> Direct Cause -> System Failure -> Ineffective Control -> Process Gap).

🔗½Corrective Actions: Specific inputs for Immediate Corrective Actions and Long-Term Preventive Actions, along with Target Completion Dates.

🔗½Official Closure Verification: Sign-off block specifying the verifier's identity, date, and final closing remarks. Updates the ticket status to `Closed`.

🔗½Personal Data Protection Act (PDPA) Consent:

- For Reporters: Displayed at the bottom of Step 3 prior to final submission.
- For HSE Managers & Superusers: Displayed at the bottom of Step 4 prior to investigation closeout.
- Ensures compliance with personal data collection and processing policies within DKSH.
- Dim and Lock Control: If the PDPA consent checkbox is not checked, the final submission action button is visually dimmed and locked (disabled) to enforce data privacy compliance before any data can be saved.

Fields involved in Step 4 (CAPA, Consent & Investigation):

No.	Field Name	Field Type	Mandatory	Description /
1	**Why 1: Direct Incident Cause**	Text Input / Text Area	Yes	Initial event description trigger.
2	**Why 2: Physical/Technical**	Text Input / Text Area	Yes	Mechanical, spatial, or physical fail-point.
3	**Why 3: Human Action / Factor**	Text Input / Text Area	Yes	Unsafe acts or process compliance issues.
4	**Why 4: Management/Process**	Text Input / Text Area	Yes	Process guidelines, checklists, or workflow gap.
5	**Why 5: Systemic Root Cause**	Text Input / Text Area	Yes	Ultimate organizational culture or resourcing gap.
6	**Immediate Corrective Action**	Text Area	Yes	Measures taken within 24 hours to contain hazard.

7	**Long-Term Preventive Action**	Text Area	Yes	Permanent process changes to prevent recurrences.
8	**Action Owner**	Text Input	Yes	Assigned supervisor responsible for implementing CAPA.
9	**Target Due Date**	Calendar Date Picker	Yes	Expected completion date for actions.
10	**Sign-off Verifier Name**	Text Input	Yes (on closure)	Lead manager validating closure (only for Level 2/3/Admin).
11	**Sign-off Closure Date**	Calendar Date Picker	Yes (on closure)	Verified closeout date.
12	**Final Closing Comments**	Text Area	Yes (on closure)	Comprehensive verification notes and checklist confirmation.
13	**PDPA Consent Checkbox**	Checkbox Selector	Yes	Consent to the processing of personal data for HSE Incident Reporting inside DKSH. Mandatory for all roles before submitting.

3.3 Unified User Journey & Incident Life Cycle Flow

Below is the structured, end-to-end user journey mapping out how an incident ticket progresses from the initial physical event to official investigation, Corrective Action (CAPA) implementation, and final administrative closure.

[INCIDENT OCCURRENCE / HSE EVENT]

```
%
%ï¿½
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%   STEP 1: BASIC OPERATIONAL DATA   %
% - Reporter inputs Date, Time & Category %
% - Selects active Distribution Center    %
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%
%ï¿½
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%   STEP 2: PERSONNEL & WITNESSES     %
% - Enters involved workers & ID cards  %
% - Logs eye witnesses & contractor tags %
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%
%ï¿½
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%   STEP 3: BODY MAPPING & CLASSIFY    %
% - Toggles anatomical injury hotspots  %
% - Checks 17-Point Compliance triggers %
% - System auto-rates Severity Class    %
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%
Is Active User a Level 1 Reporter?
%% %% % [YES: Submit Draft] %% %% %% %% %%ï¿½ [Save as Ticket Draft]
%
%% %% % [NO: Manager/Superuser] %%ï¿½
%                                     [Manager Reviews Draft]
v
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%   STEP 4: RCA 5-WHYS & CAPA DEPLOYMENT %ï¿½%% %% %% %% %% %%
% - Performs systematic 5-Whys Analysis  %
% - Commits immediate corrective action  %
% - Commits long-term preventive action  %
% - Assigns Task Owner, Deadline, Priority%
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%
%ï¿½
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%   CLOSURE VERIFICATION & SIGN-OFF    %
% - Manager verifies recording details  %
% - Submits closure date & comments    %
% - Ticket state changes: "Investigating" %
%   transitioned to "Closed" status      %
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%
%
%ï¿½
[ REGIONAL DASHBOARD UPDATES ]
Safe Days & Incident Frequency Rates sync
Executive reports generated for review
```

3.4 Module 3: My HSE Tickets Log & Search Engine

🔗 Objective: Operational database explorer for managing open tickets, reviewing drafts, and updating records.

1½Key Features:

- Text Search: Real-time fuzzy filtering across titles, locations, owners, and descriptions.
- Status Pill Indicators: Clear color coding representing ticket states:
 - `Draft` (Yellow - editable/resumable)
 - `Investigating` (Red - open under assessment)
 - `Closed` (Emerald - verified and resolved)
- Detailed Side Drawer: Fully responsive sliding panel displaying the entire incident report, rosters, body mapping, 5-Whys, and investigation outcomes without reloading the page. Includes scrollable responsive tables for Involved/Witness rosters to prevent clipping on small displays.

Fields involved in Module 3:

No.	Field Name	Field Type	Mandatory	Description /
1	**Search Filter Box**	Text Search Input	No	Real-time text filter covering descriptions, location, names, and ticket identifiers.
2	**Status Filter Tabs**	Multi-Select Group	No	Filter records selectively based on `Draft`, `Investigating`, or `Closed`.
3	**Classification Quick Filters**	Toggle / Checkbox Group	No	Filter records by `Injury`, `Ill-health`, `Property damaged`, `Near miss`, `Hazard`.

3.5 Module 4: Masters Configuration (DC Config)

 Objective: Manage organizational units (Distribution Centers) dynamically.

 Controls:

- Site Registration: Add Distribution Centers by specifying Name, Operating Country, and Assigned Manager. (Restricted to Regional HSE Manager / Superuser only).
- Activation Toggle: Easily enable or disable sites. Inactive DCs are automatically hidden from the incident reporting wizard. (Restricted to Regional HSE Manager / Superuser only).
- Role Restrictions: Country HSE Managers (Level 2) and standard Reporters (Level 1) are strictly restricted to read-only views of the Distribution Center lists and are blocked from registering new facilities or toggling statuses.

Fields involved in Module 4:

No.	Field Name	Field Type	Mandatory	Description /
1	**DC / Site Name**	Text Input	Yes	Full human-readable name of the facility/Distribution Center.
2	**Operating Country / Market**	Dropdown Selector	Yes	Target country context to assign the newly created site.
3	**Assigned Plant Manager**	Text Input	Yes	Designate the manager in charge of facility HSE compliance.

4	**DC Status Toggle**	Switch / Slider Checkbox	Yes	Active state of the DC. Non-active facilities are hidden from standard reporting drop-downs.
---	----------------------	--------------------------	-----	--

4. Non-Functional Specifications & Design Standards

4.1 Corporate Brand Identity & Design Token System

To align with the official DKSH Corporate Styling Guidelines, the application strictly employs a unified design token system for colors, typography, and visual branding:

4.1.1 Certified Color Palette

Token Category	Token Name	Hex Code	Applied System Context
Primary Brand	DKSH Crimson Red	`#BE0028`	Corporate headers, branding logos, primary titles, and high-level status anchors.
Active Accent	Active Red	`#EF233C`	High-visibility interactive elements, hover states, and primary Call-to-Actions (CTAs).
Neutral Dark	Corporate Charcoal	`#1A1A1A`	Header backgrounds, primary body text, and dark structural lines.
Neutral Light	Pure White	`#FFFFFF`	Card backgrounds, list panels, and content container sheets.
Page Background	Soft Warm Gray	`#F4F4F4`	Outer stage background to reduce physical fatigue during long operational shifts.
Section Background	Clean Slate Gray	`#F8F8F8`	Inner tables, nested headers, and alternating row backgrounds.
Digital Accents	Interactive Blues	`#90E0EF`, `#00B4D8`, `#0077B6`	Interactive hover highlights, informational micro-copy, and active links.

4.1.2 Typography & Aspect Ratios

🔗Typography Stack: High-accessibility sans-serif pairing using `Noto Sans`, `Arial`, sans-serif` to ensure global language support, pristine rendering, and strong contrast.

🔧½ Brand Logo Aspect Ratio: The DKSH brand logo is locked to an exact height of `32px` with a fluid auto-width and strict `object-fit: contain`. This completely prevents horizontal or vertical distortion while maintaining branding clear-space guidelines.

4.2 Responsive Layout Standards & Adaptive Viewport Architecture

All elements are built with a strict Mobile-First Responsive Workflow ensuring absolute visual coherence from mobile screens (<768px) to wide-screen desktop displays (>=1024px).

MOBILE VIEWPORT (<768px) - Edge-to-edge Root Layout (0px margins) - 12px Card Padding (maximizes entry area) - Single-column layout (vertical form stacking) - "One-Thumb" Bottom Navigation Bar & Selective Stepper labels	
TABLET VIEWPORT (768px - 1023px) - Centered Form Layout (max-width: 600px) - Balanced Spacing (margin: 0 auto)	
DESKTOP VIEWPORT (>=1024px) - Full-screen layout (max-width: 1440px) - Side-by-side Layout (Main Form + 320px Sidebar Draft Catalog) - Multi-column responsive grids (200px min-width inputs) - Dynamic Ribbon & Compact Panels (Labor entry ribbon, Body Parts grid)	

4.2.1 Mobile Viewport Specifications (< 768px)

- 🔧½Edge-to-Edge Root Canvas: Root container, body, and app-shell fully collapse horizontal margins and padding (`padding-inline: 0; margin-inline: 0`) to capture every available pixel of screen space.
- 🔧½Unbounded Stage Background: The grey background container (`#F4F4F4`) stretches fully edge-to-edge with `width: 100%` and zero border-radius.
- 🔧½Minimized Spacing Spacers: Inside the grey stage, outer margins and card spacers are restricted to a strict range of `8px` to `12px` (using a unified `.global-form-container` wrapper) to preserve horizontal space.
- 🔧½High-Density Form Padding: Outer form cards (using the `.global-form-card` class) reduce their internal horizontal padding to exactly `12px` (down from `32px` on desktop) to allow dense multi-column input fields (e.g. Witness Name, Roster grids) to expand to maximum legible width.
- 🔧½Progressive Stepper Disclosure: Standard labels for inactive steps are hidden in the Stepper Navigation, leaving only the active step label visible. Navigation icons are evenly distributed, and the background connecting line is locked at `top-[18px]` to perfectly bisect the icons.
- 🔧½"One-Thumb" Mobile Navigation: A fixed bottom navigation bar is loaded to allow easy single-hand thumb transitions between the Regional Dashboard, Country Dashboard, Incident Log, Report Wizard, and Masters screens.
- 🔧½Symmetric 2x2 Grid Statistics: Dashboard metrics counters (Total, Investigating, Closed, and Draft counts) reflow into a balanced `2x2` CSS grid to prevent text clipping and fit neatly within a single screen.

4.2.2 Tablet Viewport Specifications (768px - 1023px)

🔧½ Centered Form Layout: Form cards are constrained to a balanced maximum width of `600px` (`margin: 0 auto`) to avoid horizontal stretch and keep inputs comfortably grouped.

4.2.3 Desktop Viewport Specifications (>= 1024px)

- 🔧½ Maximum Width & Side-by-Side Split: Outer containers are bounded at `max-width: 1440px` with a sidebar structure. The main HSE Incident Record card expands fluidly (`flex: 1`) to fill the left column, while the "Active Drafts Catalog" anchors to a clean right-hand sidebar (`width: 320px`).
- 🔧½ Multi-Column Auto-Grid: Cramped form fields inside cards (such as Person Injured details) automatically reflow into highly legible grid structures (`grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 16px;`).
- 🔧½ High-Density Ribbon (Labor Hours Entry): The labor hours section transforms into an ultra-compact single-row "ribbon" (`flex-direction: row; align-items: flex-end; gap: 24px;` with compact `12px 20px` padding). Direct numeric input container widths are compressed to a maximum of `130px` to `140px` to optimize screen density.
- 🔧½ Grid Layout for Body Parts Affected Section: Uses a structured grid (`grid-template-columns: 280px 1fr; gap: 32px;`) to separate the scrollable Left Checklist (fixed at `280px` width) from the fluid Main Diagram Area.
- 🔧½ Side Indicators Space Optimization: The side indicator lists (Left and Right Side Indicators) are fully constrained within the card boundaries through a tight `16px` grid column gap, a streamlined central anatomical graphic (`240px` width, `480px` height), item padding reduction, and text-wrapping overrides (`white-space: normal !important`) to eliminate any horizontal overflow or out-of-frame bleed.

4.3 Common Usability, Accessibility & Interaction Rules

- 🔧½ Accessible Touch Targets: All interactive buttons, text inputs, dropdown selectors, and checklists maintain a minimum height of 48px on touch devices to comply with standard mobile usability guidelines.
- 🔧½ Collapsible Filter Accordions: Desktop and tablet filter bars are grouped into a high-contrast collapsible accordion panel, freeing up to `250px` of vertical screen height for direct dashboard data view.
- 🔧½ Tables and Dense Data Displays: All dense tabular data grids (such as Involved Rosters and Witness tables) are wrapped inside horizontal scroll wrappers (`overflow-x-auto`) with elegant trailing shadows to indicate scrollability on small screens.

5. Audit Logging & Security Compliance

- 🔧½ Central Audit Record: Each state modification (saving working hours, changing site configurations, submitting incident reports) triggers an internal record write containing the active coordinator's identifier and timestamp.
- 🔧½ Data Protection Compliance: Restricts local coordinators (`Reporter` role) from viewing other markets' safety data or accessing direct database tables.
- 🔧½ PII Protection: For basic exports or external summaries, sensitive personal information (such as personal medical logs or individual staff IDs) is automatically redacted and replaced with `[CONFIDENTIAL]` tags.

6. Operational Workflow Acceptance Matrix

To verify that the application complies with all business rules, it must pass the following scenarios:

1. REPORT	-->	2. INVESTIGATE	-->	3. ACTION-ING	-->	4. VERIFICATION
User logs draft		L2 role performs		Assign CAPA items		Superuser signs
or incident card		5-Whys diagnostics		with target dates		off and closes

1.

Wizard Draft Resume Flow: A local reporter begins logging a ticket, enters rosters, and saves it as a `Draft` at Step 2. When they click "Edit" on the Tickets Log, the wizard successfully launches and resumes exactly at Step 2.
2.

5-Whys Safety Threshold Verification: Attempting to close an incident classified as an Injury *without* completing the 5-Whys root cause analysis or assigning a Long-Term Preventive Action throws a validation error.
3.

Multi-Market Scope Isolation: Logging in as Country HSE Manager (Level 2) of Vietnam restricts the Dashboard filter and Distribution Centers dropdown strictly to Vietnam. Changing the active country context dynamically updates the UI context instantly.

7. Key User Journeys

To illustrate the real-world operational flows within the DKSH HSE CAPA System, the following user journeys define the exact paths of our primary target personas:

Journey 1: The First-Line Frontline Reporter (Level 1 Role)

 Context: A warehouse supervisor witness at a distribution center discovers a hazard or witnesses a minor injury.

[START: FRONT-LINE INCIDENT / HAZARD]

```
%  
%iŁ½  
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%  
% STEP 1: INITIAL LOGGING %  
% - Navigates to "New Ticket" tab %  
% - Inputs category, date, time & DC %  
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%  
%  
%iŁ½  
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%  
% STEP 2: ENROLL STAFF & WITNESS %  
% - Adds names, IDs, departments %  
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%  
%  
Supervisor needs missing ID?  
%% %% [YES] %% %% %iŁ½ [Save Draft (Local Storage cache)]  
%  
%  
%  
% [Search Tickets Log]  
%  
%  
%  
% [Click "Edit" on Draft]  
%  
%% %% %% %iŁ½%% %% %% %% %% %% %% %% %% %% %% %% %% %%  
%  
%iŁ½  
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%  
% STEP 3: CLASSIFY & PDPA CONSENT %  
% - Toggles affected anatomy parts %  
% - Rates severity on 17-point list %  
% - Signs mandatory PDPA checkbox %  
%% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %% %%  
%  
%iŁ½  
SUBMIT TICKET FOR INVESTIGATION ]  
- Ticket saved as "Investigating"  
- System triggers cc-notifications  
- Restricted from Step 4 access
```

1½ Flow Steps:

1. Accessing the Form: The reporter opens the application on their mobile tablet, navigating immediately to the "New Ticket" tab.
2. Context Logging (Step 1): Selects the date, time, and incident category (e.g., Near Miss), then inputs the active Distribution Center.
3. Roster Enrollment (Step 2): Adds involved persons and key witnesses to the local session roster with corresponding IDs and departments.
4. Interim Saving: Realizing they need to verify a badge ID, the supervisor clicks "Save Draft" at Step 2. The system caches all typed inputs locally and issues a success toast.
5. Resuming and Completion (Step 3): Returning later, they find their draft under "My HSE Tickets Log", click "Edit", and the wizard immediately resumes at Step 2 with all values restored. They proceed to Step 3 to describe the narrative, complete the safety checklist, sign the PDPA privacy consent directly on Step 3, and submit the ticket. (They are restricted from accessing Step 4).

Journey 2: The Country HSE Manager (Level 2 Role)

Context: An HSE Manager responsible for operations in a single country (e.g., Thailand) logs in to manage local KPIs and investigate tickets.

[illegible]

1½ Flow Steps:

1. **Local Performance Audit:** Navigates to the "Country Dashboard" to audit local statistics. The multi-market system automatically pre-filters the view specifically to Thailand.
2. **Operational Logging:** Clicks the restricted "Insert Labor Hours" action block, enters the active operational month, and logs the latest work hours to keep safety rate computations (LTIFR / TRIFR) mathematically precise.
3. **Root Cause Analysis (5-Whys):** Selects a newly submitted ticket from the "My HSE Tickets Log" and transitions to Step 4. They perform a structured 5-Whys diagnostic chain to map the systemic root failure.
4. **CAPA Assignment & Compliance Sign-off:** Defines immediate containment actions, assigns long-term preventive actions with explicit owners/target dates, and performs the final verifier sign-off to officially close out and archive the resolved ticket for their country.

Journey 3: The Regional HSE Manager / Superuser (Level 3 Role)

Context: A regional director oversees safety standards across all of Asia-Pacific and needs global configuration rights.

[illegible]

1½ Flow Steps:

1. Consolidated Overview: Navigates to the "Regional Dashboard", reviewing high-level safety trend charts and country-level comparison tables across all operational countries.
2. Master Site List Updates: Navigates to the restricted "Masters (DC Config)" tab. As a regional superuser, they possess authorization to append newly commissioned distribution centers and toggle the operational status of existing sites.
3. Compliance Sign-off: Audits all open tickets with active CAPA items, conducts final verification, and closes out the resolved investigations to archive them securely.

8. Technical Architecture & Core Data Schemas

To enable seamless, deterministic software rebuilding by automated SDLC AI engines or engineering squads, this section specifies the precise technical architecture, data model structures, state management lifecycle, and algorithmic calculation requirements.

8.1 Core Entities and TypeScript Interfaces

The system's complete business state is structured according to the following strongly typed definitions:

8.1.1 Incident Report / Ticket (`IncidentReport`)

Tracks the end-to-end lifecycle of a logged safety occurrence.

```

export enum TicketStatus {
  DRAFT = "Draft",
  INVESTIGATING = "Investigating",
  CLOSED = "Closed"
}

export enum IncidentCategory {
  INJURY = "Injury",
  ILL_HEALTH = "Ill-health",
  PROPERTY_DAMAGE = "Property damaged",
  NEAR_MISS = "Near miss",
  HAZARD_OBSERVATION = "Hazard Observation"
}

export interface RosterMember {
  id: string; // Unique UUID
  name: string;
  staffId: string;
  department: string;
  businessUnit: string;
  employeeStatus: "Direct Employee" | "Contractor";
  injuryDesignation?: "Injured" | "Involved";
  specificLocation?: string;
}

export interface IncidentReport {
  id: string; // Unique tracking ID (e.g., "TKT-2026-001")
  title: string;
  category: IncidentCategory;
  reportingDate: string; // ISO String (YYYY-MM-DD)
  reportingTime: string; // HH:MM
  dcId: string; // References DistributionCenter.id
  ccEmails: string; // Semicolon-separated email string

  // Rosters from Step 2
  involvedPersonnel: RosterMember[];
  witnesses: RosterMember[];

  // Safety Classification from Step 3
  injuredBodyParts: string[]; // List of toggled anatomical nodes (e.g. ["Knee", "Hand"])
  criteriaChecklist: number[]; // Indices of selected items from 17-point criteria (1-17)
  severityClass: "LTI" | "MTC" | "FAC" | "Near Miss" | "Hazard" | "Other";

  // Form fields for specific checklist triggers
  fatalitiesCount?: number; // Linked to Item 7
  highConsequenceCount?: number; // Linked to Item 8
  absenceCount?: number; // Linked to Item 9
  lostTimeDays?: number; // Linked to Item 9.1
  restrictedWorkCount?: number; // Linked to Item 10
  lossOfConsciousnessCount?: number; // Linked to Item 11
  medicalTreatmentCount?: number; // Linked to Item 12
  significantInjuryCount?: number; // Linked to Item 13
  firstAidCount?: number; // Linked to Item 14

  // Investigation from Step 4 (RCA & CAPA)
  why1DirectCause?: string;
  why2Technical?: string;
  why3HumanAction?: string;
  why4Management?: string;
  why5SystemicRoot?: string;
  immediateCorrectiveAction?: string;
  longTermPreventiveAction?: string;
  actionOwner?: string;
  targetDueDate?: string; // ISO String (YYYY-MM-DD)

  // Sign-off Close Metadata
  status: TicketStatus;
  signoffVerifier?: string;
  signoffClosureDate?: string; // ISO String (YYYY-MM-DD)
  closingComments?: string;

  // Governance
  pdpaConsentGiven: boolean;
  createdByEmail: string;
  createdByCountry: string; // Capitalized full name of country (e.g., "Thailand")
  createdAt: string; // ISO Timestamp
  updatedAt: string; // ISO Timestamp
}

```

8.1.2 Monthly Labor Hours Record (`LaborHoursRecord`)

Stores the operating labor exposure hours used to calculate standard frequency indexes.

```
export interface LaborHoursRecord {
  id: string; // Unique compound key: "country-year-month" (e.g., "thailand-2026-07")
  country: string; // Full capitalized country name
  year: number; // e.g. 2026
  month: number; // 1 to 12
  employeeHours: number; // Decimal support
  contractorHours: number; // Decimal support
  totalHours: number; // Derived: employeeHours + contractorHours
  updatedByEmail: string;
  updatedAt: string; // ISO Timestamp
}
```

8.1.3 Distribution Center / Site Master (`DistributionCenter`)

Defines organizational facilities available for safety logging.

```
export interface DistributionCenter {
  id: string; // Unique alphanumeric key (e.g., "DC-TH-001")
  name: string; // e.g., "Bangna Logistics Hub"
  country: string; // Full capitalized country name (e.g., "Thailand")
  plantManager: string; // Assigned supervisor
  isActive: boolean; // Toggle status
}
```

8.2 State Hydration & Storage Engine

The application manages data locally via a Local Storage Hydration Engine that simulates a cloud-hosted relational DB.

8.2.1 LocalStorage Keys Namespace

To prevent collision with other applications, the system binds strictly to the following keys:

🔑 `dksh\_hse\_reports`: JSON array of `IncidentReport`.

🔑 `dksh\_hse\_hours`: JSON array of `LaborHoursRecord`.

🔑 `dksh\_hse\_dcs`: JSON array of `DistributionCenter`.

🔑 `dksh\_active\_session`: JSON object of the current mock user profile (contains `role`, `email`, `defaultCountry`).

8.2.2 Cold-Start Boot & Seeding Logic

On initial application launch, if local storage keys are undefined or empty, the system executes a boot-strap routine to seed default data:

1. Distribution Centers: Pre-populates 5-10 core hubs across Singapore, Malaysia, Thailand, Vietnam, and Indonesia.
2. Labor Hours: Pre-populates historical monthly exposure hours for the last 12 months for each active country.
3. Sample Tickets: Injects at least 3 high-quality historical tickets:
 - 1 `Closed` ticket showing completed RCA (5-Whys), assigned CAPA, and executive sign-off comments.
 - 1 `Investigating` ticket showing a submitted incident awaiting manager investigation.
 - 1 `Draft` ticket showing a partially filled form saved at Step 2.

8.3 Mathematical Calculations & Computation Algorithms

To ensure numerical consistency across all code bases, calculations must adhere to these precise program guidelines:

8.3.1 Safe Work Days Tracker

Calculation Rule: Measures consecutive days between the current system date and the `reportingDate` of the *most recent* incident with a severity class of `LTI` (Lost Time Injury) within the active country context.

Edge Cases:

- If no LTI incidents exist in the database for the active country, the system defaults to displaying "365 Days" (as a standard compliance baseline) or another preset threshold.
- Calculated as:

$$\text{Safe Days} = \left\lfloor \frac{\text{Current Date Time} - \text{Most Recent LTI Date Time}}{24 \times 60 \times 60 \times 1000} \right\rfloor$$

8.3.2 Lost Time Injury Frequency Rate (LTIFR)

Calculation Rule: Represents the frequency of Lost Time Injuries (severity class `LTI`) per 1,000,000 exposure hours within the selected date range.

Algorithm:

1. Retrieve all incident reports for the selected country where `reportingDate` falls inside the selected start/end date range and `severityClass` === "LTI". Count the total as N_{LTI} .
2. Retrieve all labor hours records for the selected country falling within the months included in the selected date range. Sum `totalHours` as H_{total} .
3. Safe-division handler: If $H_{\text{total}} = 0$, return `0.00` to prevent division-by-zero errors.
4. Formulate:

$$\text{LTIFR} = \frac{N_{\text{LTI}} \times 1,000,000}{H_{\text{total}}}$$

8.3.3 Total Recordable Injury Rate (TRIR)

Calculation Rule: Measures total recordable injuries (LTI + Medical Treatment Cases + Restricted Work Cases) per 1,000,000 exposure hours.

Algorithm:

1. Filter incident reports where `reportingDate` is within bounds and `severityClass` is one of `["LTI", "MTC"]`. Count the total as $N_{\text{recordable}}$.
2. Compute total working hours H_{total} across those months.
3. Safe-division handler: If $H_{\text{total}} = 0$, return `0.00`.
4. Formulate:

$$\text{TRIR} = \frac{N_{\text{recordable}} \times 1,000,000}{H_{\text{total}}}$$

9. ASP.NET Web Forms 4.8 Migration & Implementation Blueprint

To support rebuilding this portal on DKSH's legacy enterprise stack, this section defines the exact architectural mappings, C# class structures, database queries, caching strategies, and user control configurations required to convert the React application into a classical ASP.NET Web Forms 4.8 (.NET Framework) website.

9.1 Multi-Project Visual Studio Solution Architecture

The migrated application must be organized as a classic N-Tier Visual Studio Solution (`DKSH.HsePortal.sln`) segmented into the following layered C# projects:

```
DKSH.HsePortal.sln
% % % 1. BusinessObjects (Class Library - POCO Domain Models)
% % % Models / (IncidentReport.cs, RosterMember.cs, LaborHour.cs)
% % % 2. DataObjects (Class Library - Data Access Layer via ADO.NET)
% % % Repositories / (IncidentRepository.cs, LaborHoursRepository.cs)
% % % DbContext / (HseDbContext.cs)
% % % 3. CachingLayer (Class Library - Memory Cache Providers)
% % % Services / (CacheService.cs wrapping System.Runtime.Caching)
% % % 4. Facade (Class Library - Orchestration & Business Logic)
% % % Controllers / (HseFacade.cs, ComplianceController.cs)
% % % 5. WebUI (ASP.NET Web Forms Web Application Project)
% % % Default.aspx (The main HSE Performance Dashboard)
% % % NewTicket.aspx (Step-by-Step Reporting Wizard with UML)
% % % TicketsLog.aspx (The central CAPA search and drawer)
% % % SiteConfig.aspx (Masters Administration)
% % % UserControls / (BodyMapControl.ascx)
% % % Startup.cs (OWIN OpenID Connect Middleware)
% % % Web.config (Application configurations and database connection)
```

9.2 Layer 1: BusinessObjects (Domain Model C# Classes)

These C# models directly translate the TypeScript schemas to preserve strict typing across the business domain:

```

namespace DKSH.HsePortal.BusinessObjects
{
    using System;
    using System.Collections.Generic;

    public enum TicketStatus
    {
        Draft,
        Investigating,
        Closed
    }

    public enum IncidentCategory
    {
        Injury,
        IllHealth,
        PropertyDamage,
        NearMiss,
        HazardObservation
    }

    public class RosterMember
    {
        public Guid Id { get; set; } = Guid.NewGuid();
        public string Name { get; set; }
        public string StaffId { get; set; }
        public string Department { get; set; }
        public string BusinessUnit { get; set; }
        public string EmployeeStatus { get; set; } // "Direct Employee" or "Contractor"
        public string InjuryDesignation { get; set; } // "Injured", "Involved", "Witness"
        public string SpecificLocation { get; set; }
    }

    public class IncidentReport
    {
        public string Id { get; set; } // "TKT-YYYY-XXXX"
        public string Title { get; set; }
        public IncidentCategory Category { get; set; }
        public DateTime ReportingDate { get; set; }
        public TimeSpan ReportingTime { get; set; }
        public string DcId { get; set; }
        public string CcEmails { get; set; }

        public List<RosterMember> InvolvedPersonnel { get; set; } = new List<RosterMember>();
        public List<RosterMember> Witnesses { get; set; } = new List<RosterMember>();

        // Step 3 Classifications
        public List<string> InjuredBodyParts { get; set; } = new List<string>();
        public List<int> CriteriaChecklist { get; set; } = new List<int>();
        public string SeverityClass { get; set; }

        // Specific form triggers
        public int? FatalitiesCount { get; set; }
        public int? HighConsequenceCount { get; set; }
        public int? AbsenceCount { get; set; }
        public double? LostTimeDays { get; set; }
        public int? RestrictedWorkCount { get; set; }
        public int? LossOfConsciousnessCount { get; set; }
        public int? MedicalTreatmentCount { get; set; }
        public int? SignificantInjuryCount { get; set; }
        public int? FirstAidCount { get; set; }

        // Step 4 RCA & CAPA
        public string Why1DirectCause { get; set; }
        public string Why2Technical { get; set; }
        public string Why3HumanAction { get; set; }
        public string Why4Management { get; set; }
        public string Why5SystemicRoot { get; set; }
        public string ImmediateCorrectiveAction { get; set; }
        public string LongTermPreventiveAction { get; set; }
        public string ActionOwner { get; set; }
        public DateTime? TargetDueDate { get; set; }

        // Sign-off
        public TicketStatus Status { get; set; } = TicketStatus.Draft;
        public string SignoffVerifier { get; set; }
        public DateTime? SignoffClosureDate { get; set; }
        public string ClosingComments { get; set; }

        // Governance
        public bool PdpaConsentGiven { get; set; }
        public string CreatedByEmail { get; set; }
        public string CreatedByCountry { get; set; }
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    }
}

```

9.3 Layer 2: DataObjects (ADO.NET Repository Implementation)

To secure raw performance under high enterprise loads on .NET Framework 4.8, the Data Access Layer (DAL) compiles commands using clean SQL queries executed via standard ADO.NET `SqlConnection` with parameter safeguards to prevent injection.

```

namespace DKSH.HsePortal.DataObjects
{
    using System;
    using System.Data;
    using System.Data.SqlClient;
    using System.Collections.Generic;
    using DKSH.HsePortal.BusinessObjects;

    public class LaborHoursRepository
    {
        private readonly string _connectionString;

        public LaborHoursRepository(string connectionString)
        {
            _connectionString = connectionString;
        }

        public LaborHoursRecord GetRecord(string country, int year, int month)
        {
            using (var conn = new SqlConnection(_connectionString))
            {
                const string query = @"
                    SELECT Id, Country, Year, Month, EmployeeHours, ContractorHours, UpdatedByEmail, UpdatedAt
                    FROM HseLaborHours
                    WHERE Country = @Country AND Year = @Year AND Month = @Month";

                using (var cmd = new SqlCommand(query, conn))
                {
                    cmd.Parameters.Add("@Country", SqlDbType.NVarChar).Value = country;
                    cmd.Parameters.Add("@Year", SqlDbType.Int).Value = year;
                    cmd.Parameters.Add("@Month", SqlDbType.Int).Value = month;

                    conn.Open();
                    using (var reader = cmd.ExecuteReader())
                    {
                        if (reader.Read())
                        {
                            return new LaborHoursRecord
                            {
                                Id = reader["Id"].ToString(),
                                Country = reader["Country"].ToString(),
                                Year = Convert.ToInt32(reader["Year"]),
                                Month = Convert.ToInt32(reader["Month"]),
                                EmployeeHours = Convert.ToDouble(reader["EmployeeHours"]),
                                ContractorHours = Convert.ToDouble(reader["ContractorHours"]),
                                UpdatedByEmail = reader["UpdatedByEmail"].ToString(),
                                UpdatedAt = Convert.ToDateTime(reader["UpdatedAt"])
                            };
                        }
                    }
                }
            }
            return null;
        }

        public void SaveRecord(LaborHoursRecord record)
        {
            using (var conn = new SqlConnection(_connectionString))
            {
                const string query = @"
                    MERGE HseLaborHours AS target
                    USING (SELECT @Id AS Id) AS source
                    ON (target.Id = source.Id)
                    WHEN MATCHED THEN
                        UPDATE SET
                            EmployeeHours = @EmployeeHours,
                            ContractorHours = @ContractorHours,
                            UpdatedByEmail = @UpdatedByEmail,
                            UpdatedAt = @UpdatedAt
                    WHEN NOT MATCHED THEN
                        INSERT (Id, Country, Year, Month, EmployeeHours, ContractorHours, UpdatedByEmail, UpdatedAt)
                        VALUES (@Id, @Country, @Year, @Month, @EmployeeHours, @ContractorHours, @UpdatedByEmail, @UpdatedAt);";

                using (var cmd = new SqlCommand(query, conn))
                {
                    cmd.Parameters.AddWithValue("@Id", $"{record.Country.ToLower()}-{record.Year}-{record.Month}");
                    cmd.Parameters.AddWithValue("@Country", record.Country);
                    cmd.Parameters.AddWithValue("@Year", record.Year);
                    cmd.Parameters.AddWithValue("@Month", record.Month);
                    cmd.Parameters.AddWithValue("@EmployeeHours", record.EmployeeHours);
                    cmd.Parameters.AddWithValue("@ContractorHours", record.ContractorHours);
                    cmd.Parameters.AddWithValue("@UpdatedByEmail", record.UpdatedByEmail);
                    cmd.Parameters.AddWithValue("@UpdatedAt", DateTime.UtcNow);

                    conn.Open();
                    cmd.ExecuteNonQuery();
                }
            }
        }
    }
}

```

9.4 Layer 3: CachingLayer (`System.Runtime.Caching` Wrapper)

To prevent redundant DB rounds for relatively static data (e.g., Distribution Center lists, historical Monthly Labor Hours), an in-memory cache is implemented:

```
namespace DKSH.HsePortal.CachingLayer
{
    using System;
    using System.Runtime.Caching;
    using System.Collections.Generic;
    using DKSH.HsePortal.BusinessObjects;

    public class CacheService
    {
        private static readonly ObjectCache Cache = MemoryCache.Default;
        private const string DcCacheKey = "DKSH_HSE_DC_LIST";

        public static List<DistributionCenter> GetDistributionCenters(Func<List<DistributionCenter>> fetchSource)
        {
            if (Cache.Get(DcCacheKey) is List<DistributionCenter> cachedList)
            {
                return cachedList;
            }

            var freshList = fetchSource();
            var policy = new CacheItemPolicy
            {
                AbsoluteExpiration = DateTimeOffset.Now.AddMinutes(30) // Expire every 30 minutes
            };
            Cache.Set(DcCacheKey, freshList, policy);
            return freshList;
        }

        public static void ClearDcCache()
        {
            Cache.Remove(DcCacheKey);
        }
    }
}
```

9.5 Layer 4: Facade (Transaction Orchestration Logic)

The Facade pattern acts as a thread-safe unified interface that coordinates data logic, validates safety calculations, and manages caching constraints:

```
namespace DKSH.HsePortal.Facade
{
    using System;
    using System.Collections.Generic;
    using DKSH.HsePortal.BusinessObjects;
    using DKSH.HsePortal.DataObjects;
    using DKSH.HsePortal.CachingLayer;

    public class HseFacade
    {
        private readonly LaborHoursRepository _hoursRepo;
        private readonly string _connectionString;

        public HseFacade(string connectionString)
        {
            _connectionString = connectionString;
            _hoursRepo = new LaborHoursRepository(connectionString);
        }

        public List<DistributionCenter> GetActiveSites(Func<List<DistributionCenter>> dbFetch)
        {
            return CacheService.GetDistributionCenters(dbFetch);
        }

        public double CalculateLtifr(int ltiCount, string country, int year, int month)
        {
            var laborHours = _hoursRepo.GetRecord(country, year, month);
            if (laborHours == null || laborHours.TotalHours <= 0)
            {
                return 0.00;
            }

            double ltifr = (ltiCount * 1000000.0) / laborHours.TotalHours;
            return Math.Round(ltifr, 2);
        }

        public double CalculateTrir(int recordableCount, string country, int year, int month)
        {
            var laborHours = _hoursRepo.GetRecord(country, year, month);
            if (laborHours == null || laborHours.TotalHours <= 0)
            {
                return 0.00;
            }

            double trir = (recordableCount * 1000000.0) / laborHours.TotalHours;
            return Math.Round(trir, 2);
        }
    }
}
```

9.6 Layer 5: WebUI & ASPX Markup Implementation

9.6.1 Web.config Configuration

Maps system connections, registers safe custom assemblies, and locks down execution:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <connectionStrings>
    <add name="HseDbConnectionString"
      connectionString="Server=tcp:sql-dksh-hse.database.windows.net,1433;Initial Catalog=HseDb;Persist Security Info=False;User ID=hse_admin;Pa
      providerName="System.Data.SqlClient" />
    </connectionStrings>
  <system.web>
    <compilation debug="false" targetFramework="4.8" />
    <httpRuntime targetFramework="4.8" />
    <pages>
      <namespaces>
        <add namespace="System.Web.Optimization" />
      </namespaces>
    </pages>
  </system.web>
</configuration>
```

9.6.2 ASPX Step-by-Step Reporting Wizard with AJAX `UpdatePanel`

To replicate the frictionless React Wizard without full-page browser updates, use ASP.NET AJAX Controls:

```

<%@ Page Title="HSE Report Wizard" Language="C#" MasterPageFile="~/Site.Master" AutoEventWireup="true" CodeBehind="NewTicket.aspx.cs" Inherits="HSE.ReportWizard" %>

<asp:Content ID="BodyContent" ContentPlaceHolderID="MainContent" runat="server">
    <div class="max-w-4xl mx-auto p-4">
        <!-- ScriptManager acts as AJAX hub -->
        <asp:ScriptManager ID="HseScriptManager" runat="server" />

        <asp:UpdatePanel ID="WizardUpdatePanel" runat="server" UpdateMode="Conditional">
            <ContentTemplate>

                <!-- STEP INDICATOR BAR -->
                <div class="flex items-center justify-between mb-6 bg-white p-4 rounded shadow-sm border border-gray-100">
                    <span class="font-bold text-xs <%= ActiveStep == 1 ? "text-red-700" : "text-gray-400" %>">1. General Info</span>
                    <span class="text-gray-300"></span>
                    <span class="font-bold text-xs <%= ActiveStep == 2 ? "text-red-700" : "text-gray-400" %>">2. Involved Roster</span>
                    <span class="text-gray-300"></span>
                    <span class="font-bold text-xs <%= ActiveStep == 3 ? "text-red-700" : "text-gray-400" %>">3. Classification</span>
                    <span class="text-gray-300"></span>
                    <span class="font-bold text-xs <%= ActiveStep == 4 ? "text-red-700" : "text-gray-400" %>">4. Root Cause (RCA)</span>
                </div>

                <!-- STEP CONTENT CONTAINERS -->
                <asp:MultiView ID="WizardMultiView" runat="server" ActiveViewIndex="0">

                    <!-- STEP 1: GENERAL INFO -->
                    <asp:View ID="Step1View" runat="server">
                        <div class="bg-white p-6 rounded-lg shadow-sm border border-gray-200">
                            <h2 class="text-lg font-bold text-slate-800 mb-4">Step 1: General Incident Specifications</h2>

                            <div class="mb-4">
                                <label class="block text-xs font-semibold text-gray-600 mb-1">Incident Title *</label>
                                <asp:TextBox ID="txtTitle" runat="server" CssClass="w-full p-2 border border-gray-300 rounded focus:border-red-600 outline-none"></asp:TextBox>
                                <asp:RequiredFieldValidator ID="rfvTitle" runat="server" ControlToValidate="txtTitle" ErrorMessage="Title is mandatory" ForeColor="red"></asp:RequiredFieldValidator>
                            </div>

                            <div class="mb-4">
                                <label class="block text-xs font-semibold text-gray-600 mb-1">Distribution Center Location *</label>
                                <asp:DropDownList ID="ddlDc" runat="server" CssClass="w-full p-2 border border-gray-300 rounded h-10 bg-white"></asp:DropDownList>
                            </div>
                        </div>
                    </asp:View>

                    <!-- STEP 2: INVOLVED ROSTER -->
                    <asp:View ID="Step2View" runat="server">
                        <div class="bg-white p-6 rounded-lg shadow-sm border border-gray-200">
                            <h2 class="text-lg font-bold text-slate-800 mb-4">Step 2: Personnel Rosters</h2>
                            <!-- Roster insertion GridView -->
                            <asp:GridView ID="gvInvolved" runat="server" AutoGenerateColumns="False" CssClass="w-full text-left text-xs border border-gray-200">
                                <Columns>
                                    <asp:BoundField DataField="Name" HeaderText="Staff Name" HeaderStyle-CssClass="bg-gray-50 p-2" ItemStyle-CssClass="p-2"></asp:BoundField>
                                    <asp:BoundField DataField="StaffId" HeaderText="Staff ID" HeaderStyle-CssClass="bg-gray-50 p-2" ItemStyle-CssClass="p-2"></asp:BoundField>
                                    <asp:BoundField DataField="EmployeeStatus" HeaderText="Category" HeaderStyle-CssClass="bg-gray-50 p-2" ItemStyle-CssClass="p-2"></asp:BoundField>
                                </Columns>
                            </asp:GridView>
                        </div>
                    </asp:View>

                </asp:MultiView>

                <!-- NAVIGATION ACTIONS PANEL -->
                <div class="flex justify-between items-center mt-6">
                    <asp:Button ID="btnBack" runat="server" Text="Previous Step" CssClass="p-2"></asp:Button>
                    <asp:Button ID="btnNext" runat="server" Text="Next Step" CssClass="p-2"></asp:Button>
                </div>

            </ContentTemplate>
        </asp:UpdatePanel>
    </div>
</asp:Content>

```

9.6.3 Code-Behind Logical Operations (`NewTicket.aspx.cs`)

Coordinates multi-view state changes asynchronously during client interactions:

```
namespace DKSH.HsePortal.WebUI
{
    using System;
    using System.Web.UI;
    using DKSH.HsePortal.BusinessObjects;

    public partial class NewTicket : Page
    {
        public int ActiveStep
        {
            get => (int)(ViewState["ActiveStep"] ?? 1);
            set => ViewState["ActiveStep"] = value;
        }

        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                ActiveStep = 1;
                WizardMultiView.ActiveViewIndex = 0;
                BindDistributionCenters();
            }

            private void BindDistributionCenters()
            {
                // Integration with Business Facade layer
                ddlDc.Items.Clear();
                ddlDc.Items.Add(new System.Web.UI.WebControls.ListItem("Bangna Logistics Hub", "DC-TH-001"));
                ddlDc.Items.Add(new System.Web.UI.WebControls.ListItem("Singapore Logistics Center", "DC-SG-001"));
            }

            protected void BtnNext_Click(object sender, EventArgs e)
            {
                if (ActiveStep == 1)
                {
                    // Validate Page logic
                    ActiveStep = 2;
                    WizardMultiView.ActiveViewIndex = 1;
                    btnBack.Visible = true;
                }
                else if (ActiveStep == 2)
                {
                    ActiveStep = 3;
                    WizardMultiView.ActiveViewIndex = 2;
                }
                WizardUpdatePanel.Update();
            }

            protected void BtnBack_Click(object sender, EventArgs e)
            {
                if (ActiveStep == 2)
                {
                    ActiveStep = 1;
                    WizardMultiView.ActiveViewIndex = 0;
                    btnBack.Visible = false;
                }
                else if (ActiveStep == 3)
                {
                    ActiveStep = 2;
                    WizardMultiView.ActiveViewIndex = 1;
                }
                WizardUpdatePanel.Update();
            }
        }
    }
}
```

9.7 Layer 6: Enterprise Identity Integrations (OWIN OpenID Connect Startup)

Enterprise SSO authenticates operations via OpenID Connect. The claims-mapping middleware intercepts context attributes to resolve role authorizations:

```
using Microsoft.Owin;
using Owin;

[assembly: OwinStartup(typeof(DKSH.HsePortal.WebUI.Startup))]

namespace DKSH.HsePortal.WebUI
{
    using System;
    using System.Security.Claims;
    using Microsoft.Owin.Security;
    using Microsoft.Owin.Security.Cookies;
    using Microsoft.Owin.Security.OpenIdConnect;

    public class Startup
    {
        public void Configuration(IAppBuilder app)
        {
            app.SetDefaultSignInAsAuthenticationType(CookieAuthenticationDefaults.AuthenticationType);

            app.UseCookieAuthentication(new CookieAuthenticationOptions());

            app.UseOpenIdConnectAuthentication(new OpenIdConnectAuthenticationOptions
            {
                ClientId = "your-dksh-azure-ad-client-id",
                Authority = "https://login.microsoftonline.com/your-tenant-id/v2.0",
                RedirectUri = "https://hse.dksh.com/signin-oidc",
                PostLogoutRedirectUri = "https://hse.dksh.com/logout",
                Scope = "openid profile email",

                Notifications = new OpenIdConnectAuthenticationNotifications
                {
                    SecurityTokenValidated = async n =>
                    {
                        var identity = n.AuthenticationTicket.Identity;

                        // Extract Active Directory claims to assign standard system roles
                        string userEmail = identity.FindFirst(ClaimTypes.Email)?.Value ?? "";

                        // Map claims dynamically to DKSH role structures
                        if (userEmail.EndsWith("@dksh.com"))
                        {
                            if (userEmail.Contains(".admin"))
                            {
                                identity.AddClaim(new Claim(ClaimTypes.Role, "Superuser"));
                            }
                            else if (userEmail.Contains(".mgr"))
                            {
                                identity.AddClaim(new Claim(ClaimTypes.Role, "Country HSE Manager"));
                            }
                            else
                            {
                                identity.AddClaim(new Claim(ClaimTypes.Role, "Reporter"));
                            }
                        }
                    }
                }
            });
        }
    }
}
```

9.8 Checklist for Production Launch in IIS Express & Windows IIS Server

To ensure zero downtime and strict adherence to DKSH corporate hosting guidelines during rebuild execution:

1. Host Configuration: Register the site as a local IIS website on Port 443 (HTTPS is mandatory for OpenID cookies). Enable ASP.NET 4.8 inside Windows Features under `Internet Information Services -> World Wide Web Services -> Application Development Features`.
2. App Pool Specifications: Provision a dedicated IIS Application Pool (`DkshHsePool`) running in Integrated Mode using the .NET CLR Version v4.0 runtime. Set `Idle Time-out (minutes)` to `0` for continuous calculator caching readiness.
3. Db Connection Encryption: Configure the database Connection String in `Web.config` with `Encrypt=True` and `TrustServerCertificate=False` to prevent intermediate packet eavesdropping inside internal DKSH networks.
4. PDPA Integrity Compliance: Implement database-level column masking on the `RosterMember.StaffId` and `RosterMember.Name` fields for draft records, aligning with Step 3's consent validation parameters.